

RinSathi (■■■■■)

Autonomous Credit & Lending Orchestrator

MID-DEFENSE HANDBOOK

Complete Guide: What We Built, How It Works,
How to Defend It & What Comes Next

TEAM MEMBERS

Unique Poudel (238017) — Team Lead

Sumina Rai (238018) — System Architect

Niraj Prasad Sah (238024) — Programmer/Developer

Yogyansh Shah (238025) — Business Analyst

Bishal Gupta (228009) — Tester

Kathmandu University School of Management (KUSOM)

Bachelor of Business Information System (BBIS) — Batch 2026

Project Deadline: July 30, 2026

CONFIDENTIAL — FOR INTERNAL TEAM USE ONLY

■ TABLE OF CONTENTS

Section	Topic	Page
1	What Is This Project? (The Big Picture)	3
2	What We Have Built — Sprint Progress Report	4
3	How The System Works — The 5-Agent Pipeline	5
4	Technical Deep-Dive: Each Agent Explained	7
5	Key Technical Decisions & Why We Made Them	13
6	Q&A Flashcard Bank — Practice Before Defense	15
7	Known Limitations & Honest Answers	19
8	■ What Comes After Mid-Defense (Final Defense Roadmap)	20
9	Quick Cheat Sheet — Glossary of Terms	22
10	Team Roles & Responsibilities	23

SECTION 1 — What Is This Project? (The Big Picture)

1.1 The Problem We Are Solving

Nepal has millions of farmers, small traders, and remittance-dependent households who need small loans. But they do not have formal pay slips or credit histories. When they walk into a bank or microfinance company, a loan officer manually checks their documents, calculates income, and makes a decision. This process is:

- **Slow** — can take days or weeks
- **Expensive** — human loan officers cost money
- **Biased** — decisions can vary from officer to officer
- **Exclusionary** — people without formal income proof are often rejected unfairly

1.2 Our Solution — RinSathi (■■■■■■■)

RinSathi means 'Loan Companion' in Nepali. It is an AI-powered software system that replaces the manual loan officer process with five intelligent software agents working together. A farmer can upload their citizenship certificate and eSewa/remittance records. RinSathi reads these documents, calculates income, checks Nepal Rastra Bank rules, and gives a loan recommendation — all in under 30 seconds.

■ **ONE-LINE SUMMARY FOR DEFENSE: 'RinSathi is an autonomous, NRB-compliant AI pipeline that processes a loan application from document upload to final decision without requiring a human loan officer.'**

1.3 Who Are We Building This For? (The Real User)

Our primary user is **Ramesh** — a farmer in Kavrepalanchok applying for a NPR 2 lakh agricultural loan. He has no formal payslips but has remittance income from a family member abroad and is a cooperative member. The old system cannot process Ramesh efficiently. RinSathi can.

SECTION 2 — What We Have Built: Sprint Progress Report

Below is an honest, detailed account of what has been **built and tested** as of mid-defense. This is what you can demonstrate live to the examiners.

Sprint	Feature	Status	Key Achievement
Sprint 1	FastAPI Backend Scaffold	■ DONE	Full project structure, routing, config
Sprint 1	Supabase/PostgreSQL Connection	■ DONE	Async connection pool, SSL, transaction pooler
Sprint 1	SharedState Pydantic Model	■ DONE	Central data contract for all 5 agents
Sprint 1	Document Agent (OCR)	■ DONE	PaddleOCR + OpenCV preprocessing pipeline
Sprint 1	Async Ticket-Poll Pattern	■ DONE	Background job processing, no timeout issues
Sprint 1	Login Frontend	■ DONE	login.html with Tailwind CSS + Alpine.js
Sprint 2a	Income Agent — eSewa Parser	■ DONE	Parses eSewa transaction JSON
Sprint 2a	Income Agent — Remittance Parser	■ DONE	Parses remittance slip data
Sprint 2a	Income Agent — Cooperative Parser	■ DONE	Parses cooperative ledger records
Sprint 2a	Income Normalization	■ DONE	Annualized irregular income to monthly estimate
Sprint 2a	Name Cross-Validation	■ DONE	Jaccard Token Similarity anti-fraud check
Sprint 2b	XGBoost Credit Scoring	■ DONE	Trained model, 886/1000 score output
Sprint 2b	SHAP Explainability	■ DONE	Plain-English audit narratives per application
Sprint 2b	Score Frontend Tester	■ DONE	Live UI showing score + repayment probability
Sprint 3	Compliance Agent (NRB Rules)	■ DONE	KYC, LTV ratio, sector cap, AML checks
Sprint 3	Decision Agent	■ DONE	Approve / Refer / Reject with reason
Sprint 3	Full Pipeline Integration	■ DONE	POST /loan/apply — all 5 agents connected
Sprint 3	Parallel Agent Execution	■ DONE	Doc + Income run concurrently via asyncio.gather

■ ALL SPRINTS 1, 2a, 2b, and 3 ARE COMPLETE. The full end-to-end pipeline is working. You can demonstrate a live loan application going through all 5 agents and returning a decision.

SECTION 3 — How The System Works: The 5-Agent Pipeline

3.1 The Big Idea — Shared State Pattern

Think of the SharedState like a **loan application form that gets passed from desk to desk**. Each agent fills in their section. No agent talks directly to another agent — they only read and write to this shared form. This design keeps all agents independent and easy to test or replace.

Step	Agent	Input	Output	Time
■	User Submits	Document image + loan details	HTTP POST request sent	~0s
①	Document Agent	Image bytes (JPG/PNG of ID card)	Extracted name, ID number, confidence score	~0.1s
①*	Income Agent	(Runs at same time as Doc Agent)	Monthly income estimate, income confidence	~0.1s
②	Score Agent	Income + doc confidence + loan details	Credit score (0-1000), SHAP explanation	~0.5s
③	Compliance Agent	Income, loan amount, sector, flags	List of compliance flags (if any)	~0.1s
④	Decision Agent	All agent outputs	Final verdict: Approve / Refer / Reject	~0.1s
⑤	API Response	Final state	JSON with decision, score, reasons	0s

■ PARALLEL vs SEQUENTIAL: Steps ① and ①* run AT THE SAME TIME (parallel) to save time — Document Agent and Income Agent do not need each other's results. Steps ②③④ run ONE AFTER ANOTHER (sequential) because each one needs the previous step's output.

3.2 The Three Possible Decisions

Decision	Meaning	When It Happens
■ APPROVE (Recommended)	Loan is recommended for approval	Credit score ≥ 0.65 (65%) AND zero compliance flags
■■ REFER	Send to a human officer for manual review	Any compliance flag found, OR score between 0.40–0.65, OR score between 0.65–0.85 AND compliance flags
■ REJECT	Loan is automatically rejected	Credit score below 0.40 (40%) AND no compliance issues

SECTION 4 — Technical Deep-Dive: Each Agent Explained

This section explains each agent in plain language. After each explanation, there is a **Defense Tip** — the one-sentence answer you should give if an examiner asks about that component.

4.1 The Data Contract — SharedState (agents/shared_state.py)

■ File: agents/shared_state.py | Library: Pydantic v2

Imagine every loan application as a **paper file folder**. When a new application arrives, a fresh empty folder is created. Each agent picks up the folder, writes their findings on it, and passes it forward. The SharedState is that folder — but in software. It is a Python class with strict rules: every field has a defined data type, and Pydantic checks every write operation.

The **validate_assignment=True** setting is the critical part. It means if the Income Agent accidentally tries to write the text 'Forty Thousand' into the income field (which expects a number), Python will immediately raise an error. This is essential in a financial system — data type mistakes could approve a loan for the wrong amount.

■ **DEFENSE TIP: 'The SharedState is a Pydantic model with validate_assignment=True. It enforces a strict data contract between agents, preventing type errors from propagating through the pipeline and causing incorrect financial decisions.'**

4.2 Document Agent (agents/document_agent.py)

■ File: agents/document_agent.py | Libraries: PaddleOCR, OpenCV (cv2)

This agent reads a photo or scan of an ID card (citizenship certificate or Lalpurja land deed) and extracts text from it. This is called **OCR — Optical Character Recognition**. Think of it as teaching the computer to read handwriting and printed text from an image.

The Preprocessing Pipeline (Why We Don't Just Feed the Raw Photo Directly)

A phone photo of a citizenship card often has shadows, wrinkles, colored ink stamps, and noise. If we feed this directly to the OCR engine, it will read garbage. So we clean it first using 4 steps:

Step	What It Does	Plain English Analogy
1. Grayscale	Remove all color, keep brightness	Photocopy the document in black & white
2. Gaussian Blur	Smooth out pixel-level noise and grain	Wipe off dust from the page
3. Otsu Binarization	Make every pixel strictly black or white	Remove ink stamps and mid-tone smudges
4. Erosion	Thicken thin character outlines	Bold the font so OCR reads it clearly

Graceful Degradation

If the image is too blurry or PaddleOCR crashes, the agent does NOT crash the whole application. Instead it sets **manual_review_required = True** and continues. This is called **graceful degradation**.

■ **DEFENSE TIP: 'The Document Agent implements graceful degradation — if OCR fails or confidence is below the 70% threshold, it flags the application for manual review instead of crashing. This ensures the pipeline always completes safely even with low-quality document uploads.'**

■■ IMPORTANT — Forged Document Question: If an examiner asks 'what if someone uploads a fake ID?', the answer is: 'The Document Agent does not verify authenticity alone. It extracts fields. Trust is established through corroboration — the name extracted from the ID is cross-checked against the eSewa account name, remittance sender name, and cooperative records. Five independent sources confirming the same identity is much harder to fake than one document.'

4.3 Income Agent (agents/income_agent.py)

■ File: agents/income_agent.py | Libraries: Python standard library, regex

This agent calculates a borrower's income from **three alternative data sources**: eSewa transaction history, remittance records from abroad, and cooperative membership ledger. This is the core innovation — traditional banks only accept payslips.

Three Income Parsers

Parser	Data Source	What It Reads
eSewa Parser	Mobile money wallet	Monthly transaction totals, frequency, account name
Remittance Parser	Bank/money transfer records	Sender amount, frequency, currency conversion to NPR
Cooperative Parser	Savings cooperative ledger	Member contributions, dividend withdrawals, balance

Name Cross-Validation (Anti-Fraud Check)

After parsing income, the agent compares the name on the ID card with the account names on the eSewa/remittance records. It uses **Jaccard Token Similarity** — a mathematical formula that compares sets of words rather than exact strings.

Example: ID says 'Bikram Prasad Shrestha'. eSewa says 'Bikram Shrestha'. Exact match? NO. Jaccard similarity? 2 matching words out of 3 total = **66% match** → **PASS**. If match is below 50%, the income confidence score is **penalized by 40%**.

■ DEFENSE TIP: 'We use Jaccard Token Similarity instead of exact string matching because names in Nepal are commonly written in shortened forms across different documents. Exact matching would flag legitimate users while Jaccard handles real-world name variations.'

4.4 Score Agent (agents/score_agent.py)

■ File: agents/score_agent.py | Libraries: XGBoost, SHAP, pandas, numpy

This is the AI/ML heart of the system. It takes all the data collected so far and calculates a **credit score** — a number from 0 to 1 that represents the probability that the borrower will repay the loan. A score of 0.88 means 88% probability of repayment.

What is XGBoost?

XGBoost stands for eXtreme Gradient Boosting. It is a machine learning algorithm that builds a 'committee of weak decision trees' — each tree is a simple yes/no question chart, but hundreds of them together become very accurate. It is widely used in banking and fintech because it handles missing data well and is fast.

What are the Input Features (What Does XGBoost Look At)?

Feature	What It Is	Impact on Score
loan_amount_npr	Size of the loan requested	Higher loan = lower score (more risk)
monthly_income_npr	Estimated monthly earnings	Higher income = higher score
income_confidence	How reliable the income data is (0-1)	Low confidence lowers score
doc_confidence	How clear the OCR reading was (0-1)	Low quality docs lower score
loan_to_income_ratio	Loan divided by annual income	Ratio above 5x is high risk

What is SHAP Explainability?

XGBoost is a 'black box' — it gives a number but doesn't explain why. SHAP (SHapley Additive exPlanations) opens this black box. It measures **how much each input feature pushed the score up or down**. The output is a plain-English explanation like: 'Monthly income of NPR 45,000 significantly increased the credit score (+12 points). High loan-to-income ratio reduced the credit score (-8 points).'

■ **DEFENSE TIP: 'SHAP explainability is legally and ethically required in credit underwriting. Nepal Rastra Bank directives mandate that automated decisions must be auditable. A black-box AI rejection is unacceptable — SHAP ensures every decision has a documented reason.'**

4.5 Compliance Agent (agents/compliance_agent.py)

■ File: agents/compliance_agent.py | No ML — pure rule-based logic

This agent does NOT use artificial intelligence. It uses **hard mathematical rules** based on Nepal Rastra Bank (NRB) Unified Directives 2080. Think of it as a compliance checklist that the system must pass before any loan decision is made.

Rule	What It Checks	Flag If Failed
KYC Completeness	Is the document clear enough? (OCR confidence > 0.9)	KYC_INCOMPLETE
Income Verifiability	Is income data reliable? (confidence > 25%)	INCOME_UNVERIFIABLE
Loan-to-Asset Ratio	Is the loan more than 75% of estimated annual income?	LOAN_TO_ASSET_BREACH
Agricultural Sector Cap	For farming loans, is amount over NPR 5,00,000?	AGRI_EXPOSURE_LIMIT
AML Flag	Is the loan amount over NPR 10,00,000? (Anti-Money Laundering)	AML_THRESHOLD

Zero Income	Is reported income zero or missing?	ZERO_INCOME_DETECTED
-------------	-------------------------------------	----------------------

■ **DEFENSE TIP:** 'We deliberately did NOT use machine learning for compliance. Regulatory rules are binary and non-negotiable. An AI model has a margin of error. Deterministic if-else rules guarantee 100% compliance with NRB directives every single time.'

4.6 Decision Agent (agents/decision_agent.py)

■ File: agents/decision_agent.py | Pure logic — reads all previous agent outputs

The Decision Agent is the final judge. It reads the outputs of all four previous agents and applies a **priority-based decision tree** with a strict order:

Priority	Rule	Verdict
1st (Highest)	Any compliance flags exist?	→ REFER to human officer immediately
2nd	Credit score is missing (calculation failed)?	→ REFER (data insufficient)
3rd	Credit score ≥ 0.65 (65% repayment probability)?	→ APPROVE (Recommend)
4th	Credit score ≥ 0.40 but < 0.65?	→ REFER (borderline case)
5th (Lowest)	Credit score < 0.40 (below safety floor)?	→ REJECT

■ **KEY INSIGHT** — Why Compliance ALWAYS Wins: 'Compliance flags override the credit score by design. A borrower can have a perfect 900/1000 credit score but if they have a SECTOR_EXPOSURE_LIMIT flag (agricultural sector loan cap exceeded), the system MUST refer them. This is non-negotiable regulatory behavior — not a choice the AI makes.'

SECTION 5 — Key Technical Decisions & Why We Made Them

Q: Why FastAPI and not Django or Flask?

FastAPI is built for high-performance async APIs. Our pipeline runs multiple agents concurrently using async/await patterns. Django is too heavy for an API-only backend. Flask doesn't support async natively. FastAPI was the right tool.

Q: Why Supabase and not a local PostgreSQL?

We are a 5-person team working from different locations. Supabase hosts our PostgreSQL database on the cloud, so every team member can connect and test without running a local database. It also provides connection pooling via port 6543.

Q: Why PaddleOCR and not Tesseract?

PaddleOCR has better support for non-Latin scripts including Devanagari (Nepali). Tesseract's Devanagari accuracy is lower on degraded document scans. PaddleOCR also provides confidence scores per character, which we use to calculate overall document confidence.

Q: Why XGBoost and not a Neural Network?

Our dataset is tabular (rows and columns of numbers), not images or text. XGBoost consistently outperforms neural networks on tabular data. It is also much faster to train, easier to explain with SHAP, and doesn't require a GPU.

Q: Why Jaccard Similarity and not Levenshtein Distance?

Levenshtein measures character-level edit distance. It would fail on 'Bikram Shrestha' vs 'Bikram Prasad Shrestha' because they differ by many characters. Jaccard treats names as sets of word tokens, so missing a middle name is just one missing word in the set — more appropriate for human name matching.

Q: Why `asyncio.gather()` for parallel execution?

The Document Agent (OCR, takes 3-8 seconds) and Income Agent (parsing, takes 1-2 seconds) have no dependency on each other. Running them sequentially would waste 1-2 seconds per application. `asyncio.gather()` runs both at the same time, reducing total pipeline time by ~1.5 seconds.

Q: Why is there a Refer verdict and not just Approve/Reject?

A binary system is dangerous. Missing data, borderline scores, or compliance edge cases should not be auto-approved or auto-rejected. The Refer verdict is a Human-in-the-Loop safety valve that routes ambiguous cases to a human loan officer. This is a standard pattern in responsible AI.

SECTION 6 — Q&A Flashcard Bank: Practice Before Defense

These are the most likely questions your examiners will ask. Read these the night before defense. Have every team member read the answers for their own component at minimum.

Q: What problem does RinSathi solve?

A: Manual loan assessment in Nepal is slow, expensive, and excludes people without formal income. RinSathi automates the entire process using AI — from document verification to final decision — in under 30 seconds, specifically for rural Nepal's alternative income earners like farmers and remittance-dependent households.

Q: What is a multi-agent system?

A: A multi-agent system is software architecture where different specialized 'agents' — each responsible for one task — work together. Like a hospital where a radiologist reads the X-ray, a cardiologist interprets the ECG, and a doctor makes the final diagnosis. In RinSathi, Document, Income, Score, Compliance, and Decision agents each handle one part of the lending assessment.

Q: What is the SharedState pattern and why did you use it?

A: SharedState is a Pydantic model that acts as a central data contract for all agents. Instead of agents communicating directly with each other (which creates tight coupling), they all read and write to this single shared data object. This keeps agents independent, testable, and replaceable.

Q: How does your system handle blurry or low-quality documents?

A: The OpenCV preprocessing pipeline applies Gaussian Blur, Otsu Binarization, and Erosion to clean the image before OCR. If the OCR confidence still falls below 70%, the system sets `manual_review_required=True` and flags the application for human review rather than crashing.

Q: What if someone uploads a forged citizenship certificate?

A: The Document Agent extracts fields for cross-validation — it is not a standalone forgery detector. Trust is established through corroboration: the name on the ID must match the eSewa account name, remittance sender name, and cooperative records. Five independent sources confirming the same identity is extremely difficult to fake simultaneously.

Q: What is Otsu Binarization?

A: Otsu Binarization is an algorithm that automatically finds the optimal threshold to convert a grayscale image into black and white. It analyzes the pixel intensity histogram and picks the threshold that maximizes the separation between text (dark) and background (light). This removes ink stamps and mid-tone smudges that confuse OCR engines.

Q: Why do you support eSewa, remittances, and cooperative records specifically?

A: These three are the dominant informal income channels in rural Nepal. Formal payslips are rare for farmers, seasonal workers, and small traders. eSewa has millions of users in Nepal. Remittances account for 25%+ of Nepal's GDP. Cooperatives serve over 30% of the rural population.

Q: What is Jaccard Similarity?

A: Jaccard Similarity measures overlap between two sets. We split names into word tokens ('Bikram Prasad Shrestha' → {bikram, prasad, shrestha}), then calculate: intersection / union. If 2 out of 3 words match, the score is 0.66 (66%). We accept matches above 50% as valid.

Q: How was your XGBoost model trained?

A: It was trained on a synthetic dataset of microfinance loan scenarios representing Nepal's rural borrower profiles. The features include loan amount, monthly income, income confidence, document confidence, and loan-to-income ratio. For production use, it would be retrained on real historical loan data from partner microfinance institutions.

Q: What is SHAP and why is it necessary?

A: SHAP (SHapley Additive exPlanations) is a mathematical framework that attributes each input feature's contribution to the model's output. It is necessary because NRB compliance requires explainable automated decisions, and because a borrower has the right to know why their loan was rejected or approved.

Q: Why is the Compliance Agent not AI-based?

A: Regulatory rules are binary and non-negotiable. The NRB loan-to-asset cap is exactly 75% — it is either breached or not. An AI model has a probability of error. Deterministic if-else rules guarantee zero error rate on compliance checks, which is the correct approach for regulatory enforcement.

Q: What NRB rules does your Compliance Agent enforce?

A: KYC completeness (document confidence threshold), income verifiability (income confidence threshold), maximum loan-to-asset ratio of 75%, agricultural sector exposure cap of NPR 5 lakh, AML threshold at NPR 10 lakh, and zero-income detection.

Q: Why does the Compliance Agent override the Credit Score?

A: This is the priority-based conflict resolution protocol. A perfect credit score does not exempt a borrower from regulatory compliance. If the agricultural sector cap is exceeded, Nepal law prohibits the loan — regardless of how creditworthy the borrower is. Compliance always wins.

Q: What is the 'Refer' verdict?

A: Refer is a Human-in-the-Loop safety mechanism. Instead of forcing a binary Approve/Reject on ambiguous cases, Refer routes the application to a human loan officer. It triggers when compliance flags exist, when the credit score is in the borderline 40-65% range, or when critical data is missing. This is responsible AI design — the system knows its own limits.

Q: Why did you run Document and Income agents in parallel?

A: `asyncio.gather()` runs both agents concurrently because they are completely independent — the Document Agent reads an image while the Income Agent reads financial transaction data. Running them in parallel saves approximately 1.5 seconds per application. The Score, Compliance, and Decision agents run sequentially because each one needs the previous agent's output.

Q: What database are you using and why?

A: PostgreSQL hosted on Supabase. PostgreSQL is an enterprise-grade relational database suitable for financial data. Supabase provides managed hosting, multi-device accessibility for our team, and a connection pooler (port 6543) that handles concurrent connections efficiently for FastAPI's async architecture.

SECTION 7 — Known Limitations & Honest Answers

Being honest about limitations shows academic maturity. Examiners respect teams that understand the boundaries of their work. Here is how to address them confidently.

■ ■ Limitation: Training Data Is Synthetic

The XGBoost model was trained on synthetic (computer-generated) loan scenarios, not real historical microfinance data.

■ **How To Answer:** Say: 'For an academic prototype, synthetic data demonstrates the architecture. A production deployment would be retrained with real historical data from our MFI partner. The pipeline architecture and feature engineering remain the same regardless of data source.'

■ ■ Limitation: OCR Accuracy on Nepali Devanagari

PaddleOCR performs well on Latin script but has variable accuracy on degraded Devanagari text from low-quality government document scans.

■ **How To Answer:** Say: 'We acknowledge this limitation. Our confidence-scoring system handles it by routing low-confidence documents to manual review rather than making decisions on bad data. Fine-tuning PaddleOCR on a Nepali-specific corpus is planned for future work.'

■ ■ Limitation: No Real eSewa/Khalti API Integration

Income data is currently parsed from mock JSON objects representing what the real API would return.

■ **How To Answer:** Say: 'Real API integration requires NDA agreements and commercial access from F1Soft (eSewa) and Khalti. Our parser is designed to the documented API schema — swapping mock data for a real API call is a one-line change in the income agent.'

■ ■ Limitation: No JWT Authentication Yet

The current system does not have user login/authentication implemented beyond the login UI.

■ **How To Answer:** Say: 'Authentication is a Sprint 4 deliverable planned for the final defense. The login frontend is built; the JWT backend middleware is next.'

■ ■ Limitation: LangGraph / CrewAI Not Used

The project proposal mentioned LangGraph/CrewAI for orchestration. We implemented a custom SharedState orchestration pattern instead.

■ **How To Answer:** Say: 'After technical evaluation, LangGraph and CrewAI added significant complexity overhead for a structured pipeline with deterministic execution order. Our custom SharedState pattern achieves the same orchestration goal with less dependency, better performance, and is easier to explain and maintain.'

SECTION 8 — ■ What Comes After Mid-Defense: Final Defense Roadmap

This section covers everything that is **NOT YET BUILT** as of mid-defense but will be presented at the final defense in July 2026. Be transparent about this with examiners — present it as a planned roadmap, not as completed work.

■ **IMPORTANT:** Do NOT claim these are done at mid-defense. If asked, say 'This is on our final defense roadmap and is planned for Sprint 4 and Sprint 5.'

Sprint 4 — Security & Authentication

- JWT Authentication:** Login system with role-based access (Admin, Loan Officer, Applicant). The login UI is built — the backend JWT middleware comes next.
- HTTPS & TLS:** Enforce encrypted connections end-to-end. Currently development uses HTTP.
- Rate Limiting:** Prevent abuse of the loan application endpoint (max N requests per minute).
- Audit Logging to Database:** Persist every agent's output to the AuditLogs PostgreSQL table for regulatory compliance and future model retraining.

Sprint 5 — Frontend Dashboard & Reporting

- Loan Officer Dashboard:** A full web UI for loan officers to review Refer-status applications, see SHAP explanations visually, approve or reject with one click.
- Applicant Status Portal:** A simple page where applicants can check their loan status using their applicant_id (the UUID from the application response).
- PDF/JSON Audit Trail Export:** Export the complete pipeline output as a downloadable PDF report (as specified in FR-05 of the SRS).
- Application History Table:** Show past loan applications in a sortable/filterable table with filtering by decision, date, sector.

Sprint 6 — Testing, UAT & Documentation

- Unit Tests (Per Agent):** Automated tests for each agent using pytest — test edge cases like zero income, missing documents, and AML breaches.
- Integration Testing:** End-to-end pipeline test with 20+ simulated loan scenarios covering all decision paths (Approve, Refer, Reject).
- User Acceptance Testing (UAT):** 5 simulated compliance officer sessions testing real-world scenarios including the 'Ramesh the farmer' primary use case.
- Final Documentation:** User manual, deployment guide, test report, and audit trail guide as required by the project SRS.
- Lalpurja Land Deed OCR:** The project proposal includes Lalpurja document support. This requires additional field templates in the Document Agent.
- Khalti API Parser:** Adding a fourth income parser for Khalti mobile wallet as an alternative to eSewa.

Final Defense Timeline (June 16 — July 30, 2026)

Period	Planned Work	Owner
--------	--------------	-------

Jun 16–30, 2026	Sprint 4: JWT auth, HTTPS, audit logging to DB	Niraj (Backend)
Jul 1–10, 2026	Sprint 5: Loan officer dashboard, applicant portal, PD, E, R	Niraj (Frontend+Backend)
Jul 11–20, 2026	Sprint 6: UAT with 20 scenarios, unit tests, integration tests	Binhas (Testing) + All
Jul 21–30, 2026	Final documentation, SRS update, deployment guide, submission	All

SECTION 9 — Quick Cheat Sheet: Glossary of Terms

Use this if you forget what a technical term means during defense.

Term	Plain English Definition
Agent	An independent software module that does one specific job (read docs, calculate income, etc.)
API	Application Programming Interface — a way for software to talk to other software over the internet
asyncio.gather()	Python function that runs multiple async functions at exactly the same time (in parallel)
AML	Anti-Money Laundering — rules to prevent illegal money from entering the financial system
Async / Await	A coding pattern that lets the server do multiple things at once without freezing
Confidence Score	A number between 0 and 1 showing how certain the system is about a reading
FastAPI	A Python web framework for building high-performance REST APIs
Graceful Degradation	When something fails, the system continues safely instead of crashing
Jaccard Similarity	A math formula that measures overlap between two sets of words (0 = no match, 1 = identical)
JWT	JSON Web Token — a secure digital pass that proves a user is logged in
KYC	Know Your Customer — the process of verifying who a borrower is (required by NRB)
LangGraph / CrewAI	AI orchestration frameworks; we replaced them with our custom SharedState pattern
LTV / LTA Ratio	Loan-to-Asset Ratio — the loan amount divided by the borrower's total estimated assets
NRB	Nepal Rastra Bank — the central bank of Nepal that sets all lending regulations
OCR	Optical Character Recognition — software that reads text from images and photos
Otsu Binarization	Algorithm that converts a grayscale image to pure black-and-white optimally
PaddleOCR	AI-powered OCR library that supports Devanagari (Nepali) script
Pydantic	Python library for data validation — ensures data types are always correct
SharedState	Our central data envelope that all 5 agents read and write to
SHAP	SHapley Additive exPlanations — explains why an AI model made a specific decision
Supabase	Cloud-hosted PostgreSQL database service used for our project database
XGBoost	eXtreme Gradient Boosting — a fast, accurate machine learning algorithm for tabular data

SECTION 10 — Team Roles & Responsibilities

Each team member should be able to explain their section confidently at the defense.

Unique Poudel (238017)	Team Lead
Project coordination, timeline management, presentation delivery, overall system narrative, stakeholder communication. Lead the opening and closing of the defense presentation.	
Sumina Rai (238018)	System Architect
System architecture design, ER diagrams, DFD Level 0/1, database schema (3NF, 14 tables), multi-agent system design decisions. Defend: 'Why did you design it this way?'	
Niraj Prasad Sah (238024)	Programmer / Developer
All backend agents (Document, Income, Score, Compliance, Decision), FastAPI setup, Supabase integration, frontend pages, pipeline orchestration, all code. Can defend any technical question about the codebase.	
Yogyansh Shah (238025)	Business Analyst
Requirements gathering (SRS document), literature review of lending platforms, business justification for design choices, user scenarios (Ramesh the farmer), NRB regulatory context. Defend: 'What business problem does this solve?'	
Bishal Gupta (228009)	Tester
Test plan, unit testing strategy, integration test scenarios, UAT plan (20 loan scenarios), bug tracking and regression testing. For mid-defense: explain the testing approach and what edge cases were tested.	

A Note From Your Senior Developer:

You have built something genuinely impressive. The five-agent pipeline, the Pydantic SharedState pattern, the SHAP explainability, the NRB rule engine — these are production-quality design decisions. Don't just memorize the answers in this book. **Understand them.** Every line of code was written for a reason. When an examiner asks 'why?', you should be able to explain it like you're explaining it to a friend at a chai pasal — in plain, confident language. You built this. Own it. Good luck to the whole RinSathi team. — ■■■■■■